

ASSIGNMENT - II

1) What are characteristics of C++ Programming Language?
 They are data members of a class which contains the values

Characteristics $\swarrow \searrow$ Static
 Nonstatic

Eg. `int i;`
`int a;`

2) What is meant by a class?

→ 1) To achieve object orientation a new data type is provided i.e. → 'CLASS'

2) Class is a user-defined datatype almost same as 'Structure' in C programming language

Class $\swarrow \searrow$ Characteristics (Data members)
 Behaviours (Functions of Class)

Eg. `Class Demo` } class name
`{ public:`
`int a;` } Characteristics
`int b;`
`void run() { }` } Behaviour
`};` } class

3) Write notes on Constructor & Destructor.

→ A) Constructor

Used to initialize characteristics & allocate resources

Types : 1) Default 2) Parametrized
 3) Copy

B) Destructor

Used to deallocate resources which were allocated inside the constructor.

Rules :

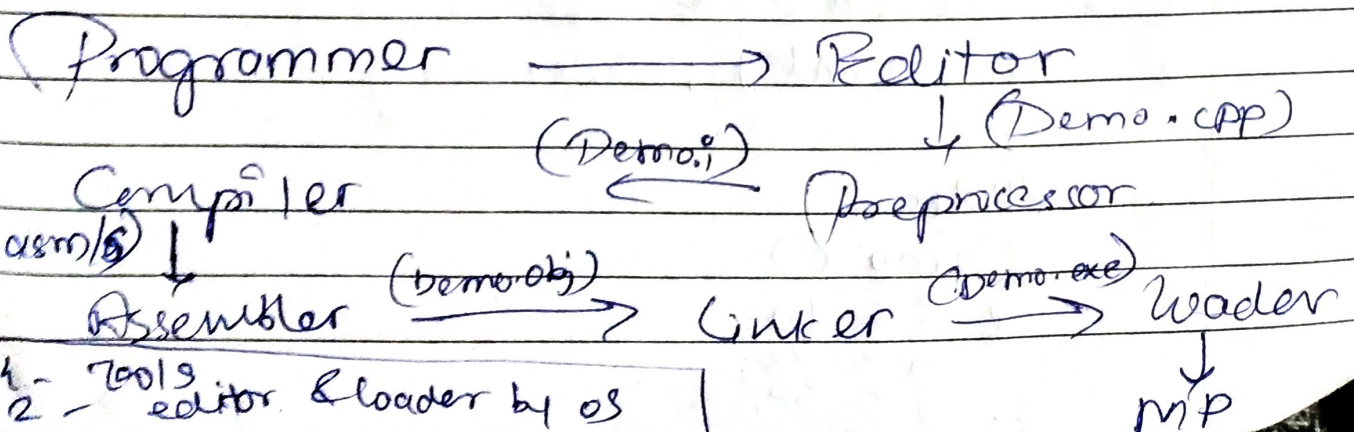
- 1) Name of Constructor $\rightarrow$ Same as class name
- 2) All constructor & destructors under public access specifier.
- 3) Should be no return type for Cons & Destr.
- 4) If we create any object w/o passing any parameters then the default constructor gets called.
- 5) Object created by passing parameters $\rightarrow$ Parameterized constructor is called.
- 6) Object created by passing another object, parameterized constructor get called.
- 7) No need of explicit call to Cons & Destr.
- 8) & operator used inside copy constructor.
 - $\hookrightarrow$ Reference operator, if not written then it may lead to recursive calls.

Q1. What are data type in C++? List them with size.

Primitive	Derived	User defined
Char (1b)	Array ^[Addⁿ of size of data member]	Structure ^[Addⁿ of size]
int (4b)	Pointer (8b)	Union ^[biggest size of member]
float (4b)	Functions	Class
double (8b)	Reference	(Add ⁿ of chars + size of Beh)
void		
Boolean (1bit)		

Q2. Explain tool chain of C++ Program.

Tool chain of C++ is same as tool chain of C.



- 6] What is meant by Abstraction?
- - Hiding something from outside world
to Achieve this we use access specifier.
- ACCESS SPECIFIERS → Who can & cannot access members (char + beh) of a class

3 Access Specifier :-

- 1] Public (All can access)
 - 2] Private (Only inside same class)
 - 3] Protected (only child can & also in same class)
-] * access.cpp
] * access1.cpp

- Irrespective of Access specifier type, memory for all char & beh gets allocated.

7] What is meant by Access Specifier? explain with example.

→ Refer Q.6]

Example:

1] Public, Private & Protected
Class Demo

```
{ public:
  int i;
```

```
private
  int a;
```

```
protected
  int j;
```

```
public:
```

```
Demo ()
```

```
{ i = 10;
```

```
  a = 20;
```

```
  j = 30; }
```


8] Write a program to find out maximum of 2 nos using procedural C & object oriented approach (C++)
 → C Program

```
#include <stdio.h>
```

```
int main()
```

```
{ int no1, no2, max;
```

```
printf("Enter 2 numbers: ");
```

```
scanf("%d %d", &no1, &no2);
```

```
max = (no1 > no2) ? no1 : no2;
```

```
printf("The maximum of %d & %d is: %d\n",  
no1, no2, max);
```

```
return 0;
```

```
}
```

C++ Program

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{ int no1, no2, max;
```

```
cout << "Enter 2 numbers: ";
```

```
cin >> no1 >> no2;
```

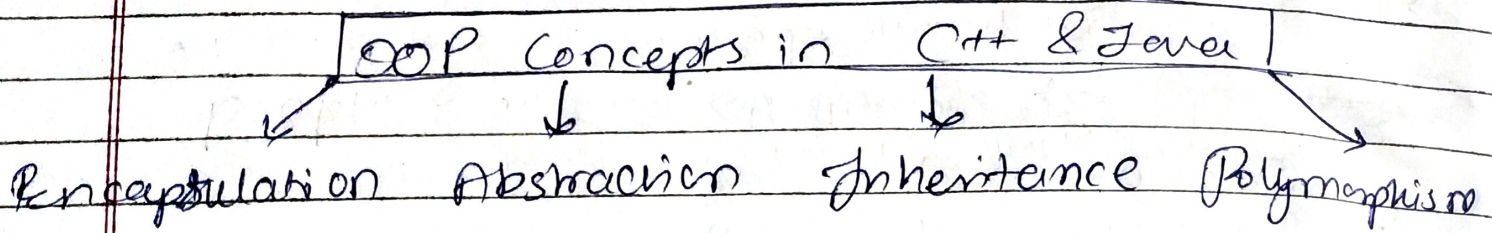
```
max = (no1 > no2) ? no1 : no2
```

```
cout << "The max of " << no1 << "and"  
<< no2 "is" : " << max << endl;
```

```
return 0;
```

```
}
```


Q1] Explain Object oriented Programming paradigm
[OOP Concepts]



1] Encapsulation :

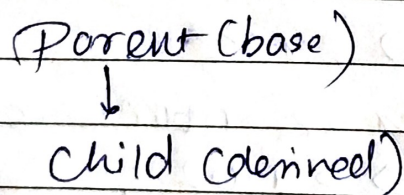
- Binding Characteristics & behaviours together
- To achieve this we create a class.

2] Abstraction :

- Hiding something from outside world
- Achieve this we use access specifier (like `private`)

3] Inheritance :

- Considered as 'reuse ability'
- Achieve this we need to inherit the derived class from parent class.



4] Polymorphism : (3 types)

- Single name multiple behaviours

- a) Compile time - Overloading
- b) Run time - Overriding

10] Write down the difference between C and C++

Aspect	<u>C</u>	<u>C++</u>
Paradigm	Procedural PL	Object oriented PL
Primary Use	System Level Programming (e.g. OS, drivers)	System & Application level programming (e.g. games, GUIs)
Class	Not supported	Fully supports classes & objects
Functions	must be declared before use	Can be defined anywhere.
Data Security	Limited (no direct mechanism for data hiding)	Enhanced with encapsulation & Access Specifier.
Memory management	Uses : malloc(), calloc() & realloc(),	Uses new & delete operators for dynamic memory management
Exception Handling	Not supported	Supported using - try, catch and throw.
File extension	. C	. cpp